

CHAPTER 5: SOFTWARE DESIGN AND ARDUINO CODE EXPLANATION

5.1 Introduction

Software is the intelligence of the Smart Irrigation System. The software is responsible for reading sensor data, processing values, displaying information, and controlling irrigation automatically.

In this project, the software is developed using:

- Arduino IDE
- Embedded C/C++
- ESP32 Board Package
- Adafruit Libraries
- DHT Library

The program continuously monitors environmental conditions and makes irrigation decisions based on soil moisture values.

5.2 Software Development Environment

Arduino IDE

Arduino IDE is used to write, compile, and upload programs to ESP32.

Functions of Arduino IDE

- Writing source code
- Compiling code
- Uploading code to ESP32
- Monitoring serial output
- Debugging

Advantages

- Free and Open Source
 - User Friendly
 - Large Community Support
 - Library Support
-

5.3 Required Libraries

The following libraries are required for the project.

1. DHT Library

```
#include <DHT.h>
```

Purpose

Reads temperature and humidity from DHT11 sensor.

Functions Used

```
dht.readTemperature();  
dht.readHumidity();
```

2. Wire Library

```
#include <Wire.h>
```

Purpose

Provides I2C communication between ESP32 and OLED display.

Why Needed?

OLED uses I2C protocol.

I2C Pins:

SDA → GPIO21

SCL → GPIO22

3. Adafruit GFX Library

```
#include <Adafruit_GFX.h>
```

Purpose

Provides graphics functions.

Used for:

- Text display
 - Shapes
 - Cursor control
-

4. Adafruit SSD1306 Library

```
#include <Adafruit_SSD1306.h>
```

Purpose

Controls OLED display.

Functions:

- Display initialization
 - Display updates
 - Text output
-

5.4 Pin Definitions

```
#define DHTPIN 4  
#define DHTTYPE DHT11  
  
#define SOIL_PIN 34  
#define RELAY_PIN 26
```

Explanation

GPIO4

Connected to DHT11 DATA pin.

GPIO34

Connected to Soil Sensor AO pin.

GPIO26

Connected to Relay IN pin.

5.5 OLED Configuration

```
#define SCREEN_WIDTH 128  
#define SCREEN_HEIGHT 64
```

Purpose

Defines OLED resolution.

Width = 128 pixels

Height = 64 pixels

OLED Object:

```
Adafruit_SSD1306 display(  
  SCREEN_WIDTH,  
  SCREEN_HEIGHT,  
  &Wire,  
  -1);
```

Purpose

Creates OLED display object.

5.6 Sensor Object Creation

```
DHT dht(DHTPIN, DHTTYPE);
```

Purpose

Creates DHT11 sensor object.

Allows access to:

```
dht.readTemperature();  
dht.readHumidity();
```

5.7 Calibration Values

```
#define AIR_VALUE 4095  
#define WATER_VALUE 1200
```

Purpose

Used for converting raw soil sensor values into percentage.

Calibration Procedure

Sensor in Air:

Reading $\approx$ 4095

Sensor in Water:

Reading $\approx$ 1200

These values form the calibration range.

5.8 Setup Function

The setup() function executes only once after ESP32 powers ON.

```
void setup()
{
}
```

Serial Communication

```
Serial.begin(115200);
```

Purpose

Starts Serial Monitor.

Used for debugging.

Baud Rate:

115200

Initialize DHT11

```
dht.begin();
```

Purpose

Starts DHT11 sensor.

Without this command sensor data cannot be read.

Configure Relay Pin

```
pinMode(RELAY_PIN, OUTPUT);
```

Purpose

Sets GPIO26 as output.

ESP32 will control relay using this pin.

Keep Pump OFF at Startup

```
digitalWrite(RELAY_PIN, HIGH);
```

Reason

Relay is Active LOW.

HIGH = Relay OFF

LOW = Relay ON

Prevents accidental pump activation.

Initialize OLED

```
display.begin(  
  SSD1306_SWITCHCAPVCC,  
  0x3C);
```

Purpose

Starts OLED display.

Address:

0x3C

Display Settings

```
display.setTextSize(1);  
  
display.setTextColor(WHITE);
```

Purpose

Controls text appearance.

5.9 Loop Function

The loop() function runs continuously.

```
void loop()  
{  
}
```

All monitoring and irrigation decisions occur here.

5.10 Reading Temperature

```
float temperature =  
dht.readTemperature();
```

Purpose

Reads current temperature.

Example:

32°C

5.11 Reading Humidity

```
float humidity =  
dht.readHumidity();
```

Purpose

Reads relative humidity.

Example:

47%

5.12 Reading Soil Moisture

```
int rawValue =  
analogRead(SOIL_PIN);
```

Purpose

Reads raw ADC value from sensor.

Example:

1535

5.13 Moisture Percentage Calculation

```
int moisturePercent =  
map(rawValue,  
AIR_VALUE,  
WATER_VALUE,  
0,  
100);
```

Purpose

Converts raw value into percentage.

Example:

1535 → 88%

Constrain Function

```
moisturePercent =  
constrain(  
moisturePercent,  
0,  
100);
```

Purpose

Limits values between:

0% and 100%

Avoids incorrect readings.

5.14 Pump State Variable

```
bool pumpState = false;
```

Purpose

Stores pump status.

false = OFF

true = ON

5.15 Irrigation Decision Logic

```
if(moisturePercent < 40)
{
    digitalWrite(RELAY_PIN, LOW);
    pumpState = true;
}
else
{
    digitalWrite(RELAY_PIN, HIGH);
    pumpState = false;
}
```

Working

If moisture < 40%

Pump ON

If moisture $\geq$ 40%

Pump OFF

Example

Soil Moisture = 25%

Pump ON

Soil Moisture = 88%

Pump OFF

5.16 Serial Monitor Output

```
Serial.print("Temperature:");  
Serial.println(temperature);
```

Purpose

Displays values on Serial Monitor.

Output Example:

```
Temperature: 32.10 C  
Humidity: 47.00 %  
Raw Soil: 1535  
Moisture: 88 %  
Pump: OFF
```

5.17 OLED Display Logic

Clear Display

```
display.clearDisplay();
```

Move Cursor

```
display.setCursor(0,0);
```

Print Temperature

```
display.print("Temp:");  
display.print(temperature);
```

Print Humidity

```
display.print("Hum:");  
display.print(humidity);
```

Print Soil Moisture

```
display.print("Soil:");  
display.print(moisturePercent);
```

Print Pump Status

```
display.print("Pump:");
```

Update OLED

```
display.display();
```

Purpose

Sends data to OLED screen.

Without this command OLED will not refresh.

5.18 Delay Function

```
delay(2000);
```

Purpose

Waits 2 seconds before next reading.

Advantages:

- Stable readings
- Reduced processor load
- Better display visibility

5.19 Program Flowchart

START

↓

Initialize ESP32

↓

Initialize Sensors

↓

Initialize OLED

↓

Read Temperature

↓

Read Humidity

↓

Read Soil Moisture

↓

Calculate Moisture %

↓

Display Values

↓

Moisture < 40 ?

↓

YES → Pump ON

NO → Pump OFF

↓

Repeat Forever

5.20 Algorithm

Step 1: Start System

Step 2: Initialize OLED

Step 3: Initialize DHT11

Step 4: Configure Relay

Step 5: Read Sensors

Step 6: Calculate Moisture %

Step 7: Display Data

Step 8: Compare Threshold

Step 9: Control Relay

Step 10: Repeat

5.21 Advantages of Software Design

- Simple Logic

- Easy Maintenance
 - Low Memory Usage
 - Fast Execution
 - Modular Design
 - Expandable for Wi-Fi Integration
-

5.22 Future Software Enhancements

Future versions may include:

- Blynk Mobile App
 - ThingSpeak Cloud
 - MQTT Communication
 - Weather Forecast API
 - AI-Based Irrigation
 - Remote Monitoring Dashboard
-

5.23 Summary

This chapter explained the complete software design of the IoT Smart Irrigation and Plant Monitoring System. The Arduino program reads sensor data, calculates moisture percentage, updates the OLED display, and automatically controls irrigation through a relay-operated water pump. The software architecture is simple, efficient, and suitable for future IoT enhancements.